

Apuntes_NumPy_1_Arrays_Explicacion_y_Funciones

May 11, 2026

1 Apuntes NumPy 1: Que es esta libreria y sus objetos *Arrays*

1.1 ¿Que es NumPy?

Como hemos visto ahora, la versatilidad y la capacidad de Python como lenguaje de programación que ayude a resolver situaciones a nivel de usuario son enormes y muy poco limitadas.

Uno de los principales usos que se implementaron en su comienzo es el uso del lenguaje como herramienta para facilitar el **calculo numerico científico** que necesita de complicadas funciones de computacion para los calculos vectoriales y matriciales.

Esta herramienta que vamos a conocer a lo largo de este modulo que nos permite desarrollar estas operaciones, compuesta por diferentes tipos de datos y funciones para el álgebra lineal y cálculo vectorial y matricial se denomina:

NumPy (*Numeric Python*)

NumPy es una clase compuesta por una serie de atributos (**tipos de datos**) y metodos que facilitan la capacidad de computar datos para calculos vectoriales y matriciales.

Al ser una libreria externa siempre sera necesario hacer uso de ***import*** para traer la libreria a nuestro archivo y operar con él.

import numpy as np

TIP: Siempre se utiliza el alias ***np*** por convencion.

Estas son algunas de sus características:

- Es la libreria de computacion cientifica la que mas posibilidades, mas informacion y mas flexibilidad nos da al trabajar con datos.
- Existen hasta **24 tipos de datos (*scalars*)** en esta libreria. Los *Arrays* es el mas conocido y aprenderemos a utilizar mejor en esta unidad.

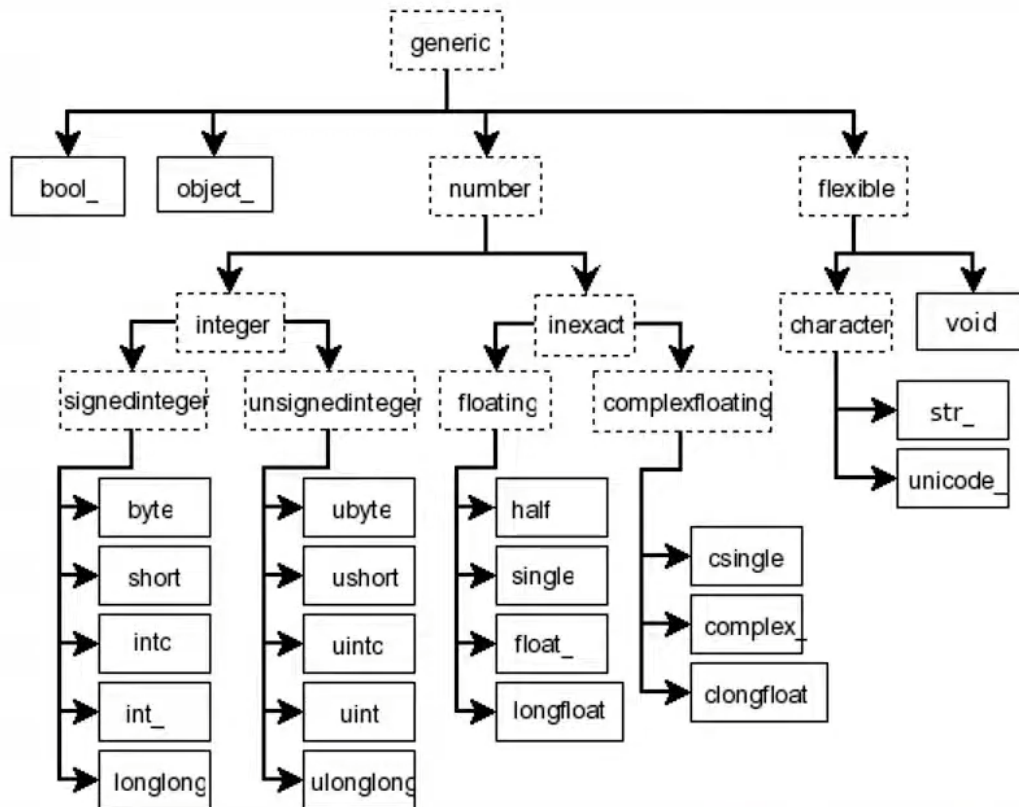
Los ***Scalars*** son objetos de Numpy que heredan de *np.generic* y sirve para representar cada uno de los valores que gestionamos

Estos *Scalars* **introducen una flexibilidad mayor en la gestion de datos**

Estos *scalars* los utilizaremos **dentro de los *Arrays***

Algunos de estos son heredados de Python, *primarios*, (*int*, *float*, *str*) pero otros son implementados nuevos (*bool* o *datetime*)

A continuacion se vera una imagen que incluye estos tipos de datos



1.2 Arrays: ¿Que son?

Los **Arrays** son el tipo de dato basico con el que se trabaja en esta libreria.

Son objetos que representan vectores o matrices **multidimensionales** en las que todos los elementos **son de un tipo**, númerico generalmente.

Estos objetos son denominados **ndarray**

Los **Arrays** nos van a dar la posibilidad de realizar operaciones de manera muy sencilla con estructuras de datos mas complejos (*scalars*).

En Python *basico* existen las listas que nos permite minimamente operar pero en cuanto requiere trabajar funciones en álgebra avanzado, no estan diseñadas para poder comportarse como necesitamos, de ahí que nazcan los **Arrays**

1.2.1 Diferencias entre **Arrays** y Listas en Python

Antes de continuar, es importante hacer esta diferenciacion entre estos 2 tipos de estructuras de datos.

- La 1º diferencia es que las listas pueden estar compuestas por **datos heterogeneos** y los **Arrays** deben ser **datos homogeneos** (del mismo tipo, *scalar*)

`lista = ["carlos", 5.4, -6, "P"]`

`ndarray = ([5, 7, 89]) / ndarray = (["carlos", "p"]) / ndarray = ([5.4, 7.7, 89.0])`

Esto es así para mejorar el rendimiento del uso de este tipo de dato

- La 2º es la capacidad de realizar **operaciones matemáticas más eficientes**, debido a varios motivos:
 - Cuando realizas un cálculo en una lista, el programa **la recorre** para realizar el cálculo, pero en los ***arrays** no la recorre** solo se realiza como una unidad.
 - el *array* consume menos memoria e incluye referencias al tipo de dato

Es debido, por así decirlo, a que *Python Básico* almacena en su memoria “el valor del dato” + “el tipo de dato” **para cada dato** mientras que *NumPy* almacena “valor del dato” + “un tipo de dato” **para todos los datos del *Array***

- La 3º es que se pueden **crear y manipular *Arrays* de diferentes formas**

Pueden venir desde una lista, podemos generar que dispongan de varias dimensiones o de 1 única dimensión, introducir tuplas...

TIP: Se sabe el nº de dimensiones de una matriz **por el número de []** que recogen el array

1.2.2 Atributos de Array

Los Array son objetos que creamos y por tanto disponen de atributos que los caracterizan y componen.

Estos atributos pueden ser convocados para cada Array

Aquí se indican los distintos atributos:

- ***.shape***: Nos indica el **nº de elementos en cada dimensión** del Array

Es decir en una matriz de 2 dimensiones y 3 filas, nos indicará un resultado de (2,3)

- ***.size***: Nos indica el **nº de elementos** del array
- ***.dtype***: Nos indica el **tipo de elementos** del array

Es posible indicar en el array el tipo de dato mediante la siguiente sentencia:

“nombre array” = np.array([“elementos del array”], dtype = “tipo de scalar”)

- ***.ndim***: Nos indica el **nº de dimensiones** del array

1.3 Arrays: ¿Cómo se interacciona con ellos?

Una vez visto que son los Arrays, necesitamos entender **que comportamientos tienen para poder utilizarlos**

Existen 3 conceptos que debemos conocer para poder utilizarlos con los métodos:

- ***Indexación***
- ***Iteración***
- ***Slicing***

Indexación de Arrays Este concepto recoge la **capacidad de acceso a los datos**, en este caso del Array.

Es el *índice* de estas estructuras y siempre cumplirán estas reglas:

- Empiezan en 0 y terminan en n-1.

Al igual que en las listas, tuplas o diccionarios, el conjunto de elementos que se componen **tienen posiciones** que vienen desde el 0 al n-1

- Cuando existen disponemos de *varias dimensiones* debemos de ubicar en primer lugar la dimension (o dimensiones) y por ultimo el índice del elemento al final.

Ejemplo: En este Array de 2 dimensiones

ndarray = ([2,0,3], [4,6,8])

Para ubicar la fila donde se ubica el 0 y el 6, lo indicaremos como (1,1) , ya que se ubica en la posición 1 de la dimension, en la fila 1

Iteración de Arrays Este concepto se entiende como **el proceso que se realiza para pasar por todos los elementos del Array**

Generalmente no se necesario utilizarlos en los Array ya que ralentiza el rendimiento del programa

Tiene estas características:

- Se utiliza el bucle *for*
- Si queremos acceder a cada elemento **utilizaremos anidación de bucles *for***

Utilizaremos tantos bucles *for* anidados por cada dimension que tenga el Array

Ejemplo de expresión para 2 dimensiones:

for i in "nombre array": - acceso a las filas

for j in i: - acceso a las columnas

```
[1]: import numpy as np

my_array = np.random.random((4,4))
print(my_array)

[[0.22575734 0.55265385 0.6247295  0.79764049]
 [0.0797322  0.40752031 0.12209789 0.99104098]
 [0.15078313 0.59190748 0.62326074 0.26488884]
 [0.51129975 0.07614997 0.08449661 0.93593383]]
```

Se puede hacer una valoración mediante un *if* o mediante una *máscara booleana*

```
[2]: import numpy as np

my_array = np.random.random((4,4))
for i in my_array:
```

```

for j in i:
    if j > 0.5:
        print (f" {j} es mayor a 0.5")

```

```

0.6102728242196804 es mayor a 0.5
0.8966636562293838 es mayor a 0.5
0.6336649237822757 es mayor a 0.5
0.6514017051571042 es mayor a 0.5
0.5003498400334113 es mayor a 0.5
0.883285744989074 es mayor a 0.5

```

```

[6]: import numpy as np

my_array = np.random.random((4,4))
for i in my_array:
    for j in i:
        print(j)
print (my_array [my_array > 0.5])

```

```

0.6956781743544672
0.7515500907704447
0.5943971606899808
0.7940695290234246
0.3847230061602316
0.09738144463530818
0.9204488075652574
0.025679190723806755
0.041851514110980204
0.8305252538863691
0.9267289981723918
0.8116192559534954
0.24773139049011583
0.21551936720544462
0.8454026270963325
0.6855412147971818
[0.69567817 0.75155009 0.59439716 0.79406953 0.92044881 0.83052525
 0.926729   0.81161926 0.84540263 0.68554121]

```

Slicing de Arrays Este concepto se entiende como el **proceso de obtención de una parte del Array**

Este comportamiento de los array, es de los mas interesantes en gestion de datos ya que nos van a permitir:

- Fraccionar cantidades ingentes de datos de manera automatica, permitiendo que haya cambios de criterios en las operaciones
- Obtener resultados de una prueba de calculo para entreno de modelos de prediccion

Se utiliza para ello los : ya que permite generar *la rebanada*

La `,` sirve para seleccionar elementos concretos del array escoger, por fila, columna,...

Los `:` sirven para escoger todos los elementos del array por fila, columna,...

Tiene estas características:

- Siempre se define de **un índice a otro índice**
- Se puede realizar mediante el uso de corchetes `[]`
- Podemos seleccionar los elementos exactos indicando **la fila y la columna** mediante el uso de las comas `,`

Siempre se indicara el índice así *“nombre array”[“Índice de fila” , “índice de columna”]*

- Podemos utilizar los elementos *start*, *stop* y *step*

Ejemplo solo uso de *start*: *“nombre array”[“índice”:]*

Ejemplo solo uso de *stop*: *“nombre array”[:“índice”]*

Ejemplo solo uso de *start*, *stop*: *“nombre array”[“índice” : “índice”]*

Ejemplo solo uso de *start*, *stop* y mas dimensiones: *“nombre array”[“índice” : “índice” , “índice”]*

Ejemplo solo uso de *start*, *stop* y *step*: *“nombre array”[“índice” : “índice”:“índice step”]*

Ejemplo podemos elegir **mas filas** utilizando las comas: *“nombre array”[[“índice fila”, “índice fila”] , :]*

Ejemplo podemos elegir **mas columnas** utilizando las comas: *“nombre array”[: , [“índice columna”, “índice columna”]]*

- Podemos utilizar *mascaras booleanas* para elegir que elementos queremos quedarnos

Devolvera un Array de 1 dimension con los que valores *True*

Mascaras Booleanas con los Arrays Existe otra funcionalidad con los Arrays que nos va a permitir realizar validaciones muy basicas.

Son las llamadas *mascaras booleanas*

Se obtienen cuando en un Array entero, con cualquier forma, **compuesto por valores**, lo comparamos con una expresion **simple de evaluacion**

variable = np.array ([“valores enteros”])

variable “=” / “!=” ”valor entero buscado

Nos devolvera un Array **con la misma forma** (vector, 2 dimensiones,...) en el que indicara **que valor cumple (*True*) o no cumple (*False*) la evaluacion**

1.3.1 Funciones para usar Arrays

Existen funciones específicas que podemos utilizar con estas estructuras que nos van a dar la posibilidad de sacarle el potencial que hemos mencionado.

Existen 3 grandes grupos de funciones que permiten trabajar con arrays: **Generacion** , **Modificacion**, **Division**

Funciones de generacion de arrays Seran aquellos metodos que nos permitiran generar arrays de distintas formas

- ***np.ones()***: Genera un Array con un tamaño y tipo indicados, con valores 1.
- ***np.zeros()***: Genera un Array con un tamaño y tipo indicados, con valores 0.
- ***np.full()***: Genera un Array con un tamaño y tipo indicados, con el mismo valor para todos los elementos.
- ***np.eye()***: Genera un Array identidad.
- ***np.arange()***: Genera un Array de 1 dimension (vector) de un intervalo dado
- ***np.linspace()***: Genera un Array de 1 dimension (vector) con un intervalo dado y valores igualados, ambos limites incluidos

Funciones de libreria **random**: Nos permitira hacer diferentes funciones con numeros aleatorios.

Funciones .random La libreria **random** se aloja dentro de la libreria numpy de modo que podemos utilizar todos sus metodos y podemos genera objetos.

Algunos de los metodos que vamos a ver son:

- ***np.random.random()***: Genera un Array, con valores aleatorios entre 0.0 y 1.0.
- ***np.random.uniform()***: Genera un Array, con distribucion uniforme entre 2 valores indicados, con un tamaño específico
- ***np.random.normal()***: Genera un Array, con distribucion normal (media=0, desv.tipica=1), con un tamaño específico
- ***np.random.choice()***: Genera un Array, vectorial, en base a los valores de otro array, eligiendo valores aleatorios con unas condiciones.

Funcion *np.random.random()* Genera un Array, con valores aleatorios entre 0.0 y 1.0.

```
[34]: import numpy as np

my_array = np.random.random(7)
print(my_array)
```

```
[0.9501295  0.23030288 0.54848992 0.90912837 0.13316945 0.52341258
 0.75040986]
```

Funcion *np.random.uniform()* y *np.random.normal()* Genera un Array con valores aleatorios, con distribucion uniforme entre 2 valores indicados, con un tamaño específico.

En el caso de ***np.random.uniform()*** los valores aleatorios estaran comprendidos entre indicados

En el caso de `np.random.normal()` los valores aleatorios tendran una distribucion **normal**

Se compone:

- *start* valor inicial
- *stop* valor final
- *size* = (*dimensiones*, *filas*) como sera la matriz resultante

```
[35]: import numpy as np

my_array = np.random.uniform(2, 45, size=(2,6))
print(my_array)

[[30.76756936 22.11337297 10.80851088 23.10293323 18.01254164 22.52824966]
 [17.73328659 38.03047376 35.05184278 15.50177112 26.6228893 13.87010908]]
```

```
[36]: import numpy as np

my_array = np.random.normal(0, 1, size=(2,6))
print(my_array)

[[-2.0644148 -0.66215934 -1.20421985 1.46197563 1.76616088 -0.32941375]
 [ 0.84073324 -0.1799864 0.56806189 -0.7528372 -1.7083392 -1.80309866]]
```

Funcion `np.random.choice()` Genera un Array con valores aleatorios a partir de otro Array creado previo, permitiendole elegir el nº de valores y si se pueden repetir o no.

Solo puede hacerse sobre vectores

Se compone por tanto:

- 1º El Array del que extraera la muestra aleatoria
- 2º N° de valores que escogera y compondran el Array
- 3º La posibilidad de coger valores repetidos

nombre array = `np.random.choice` ("*nombre array del que cogera datos*",
size = "*nº de valores*", *replace* = (*False** si no se repiten / *True* si se repiten))*

```
[41]: import numpy as np

my_array = np.random.random(45)
my_array2 = np.random.choice(my_array, size=5, replace=False)
print(my_array2)

[0.27219584 0.49847177 0.26758796 0.05897287 0.72127594]
```

Funcion `np.ones()` y `np.zeros()` Genera un array de *inicializacion a unos* que genera un array con valores 1.

En el caso de `np.zeros()` es con valores 0

En el caso de `np.random.random` es con valores pseudoaleatorios entre 0 y 1.

Este(os) Arrays(s) se compone(n) de 2 elementos que los definen:

- 1º **el tamaño** o *shape* nos indica la forma del array, si dispone de 1 dimension o de mas dimensiones (utilizando las tuplas). > Si indicamos un unico valor lo considerara **un vector** con 1 dimension > > Si indicamos una tupla, lo considerara **una matriz**, cada valor en la misma indicara, **numero de filas** y **numero de valores en la fila** (o *scalars*) > > > ejemplo: (2,4) = 2 Filas, 4 Valores por fila
- 2º **el tipo** o *dtype* nos indica el tipo de dato del array, por defecto sera `numpy.float64`

```
[5]: import numpy as np

my_array = np.ones(7, int)
print(my_array)
```

```
[1 1 1 1 1 1 1]
```

```
[9]: import numpy as np

my_array = np.zeros(7, int)
print(my_array)
```

```
[0 0 0 0 0 0 0]
```

Funcion `np.full()` Genera un Array definido con tupla (**solo genera matrices**) y hara que todos los elementos tengan el mismo valor dado

Se compone entonces:

- 1º Definicion de la matriz: Indicando el nº de dimensiones y filas
- 2º El valor que compondra todos los elementos de la matriz

```
[28]: import numpy as np

my_array = np.full((2,4),7)
print(my_array)
```

```
[[7 7 7 7]
 [7 7 7 7]]
```

Funcion `np.eye()` Genera un Array con un nº filas dado, que generara unos valores en *identidad* (valor 1 en el principio, final y en la diagonal, el resto valores 0)

Se compone entonces:

- Del numero de filas que componen la matriz.

```
[29]: import numpy as np

my_array = np.eye(7)
```

```
print(my_array)
```

```
[[1. 0. 0. 0. 0. 0. 0.]  
 [0. 1. 0. 0. 0. 0. 0.]  
 [0. 0. 1. 0. 0. 0. 0.]  
 [0. 0. 0. 1. 0. 0. 0.]  
 [0. 0. 0. 0. 1. 0. 0.]  
 [0. 0. 0. 0. 0. 1. 0.]  
 [0. 0. 0. 0. 0. 0. 1.]]
```

Funcion *arange()* Generara un Array de 1 unica dimension y un rango de numeros (*int* o *float*).

Caracteristicas:

- *start* sera el inicio del intervalo (por defecto 0)
- *stop* final del intervalo (*int* o *float*)
- *step* indica la diferencia entre 2 valores del intervalo (por defecto 1)

```
[4]: import numpy as np  
  
my_array = np.arange(2,10,7)  
print(my_array)
```

```
[2 9]
```

Funcion *linspace()* Similar a la funcion, *arange()*

Generara un Array de 1 unica dimension, incluyendo los numeros limite, generando **un espaciado entre los valores igualado**.

Es decir, si le indicas (2,10,3), generara un Array que incluya el 2 y el 10 e incluire un 3º numero, para que sea un Array de 3.

Esto puede implicar que el intervalo este obligado a coger numeros float para conseguirlo

Caracteristicas:

- *start* sera el inicio del intervalo (por defecto 0)
- *stop* final del intervalo (*int* o *float*)
- *nº de valores* indica los numeros que compondran el total del Array y obligara a la funcion a ceñirse para hacer el intervalo

```
[10]: import numpy as np  
  
my_array = np.linspace(2,10,7)  
print(my_array)
```

```
[ 2.          3.33333333  4.66666667  6.          7.33333333  8.66666667  
 10.         ]
```

Funciones de modificacion de arrays Seran aquellos metodos que permitiran generar otro array distinto con sus mismos elementos, en base a uno ya existente

- **.reshape()**: Modifica un Array existente y te devuelve otro con otra forma. > **.T**: Para transponer filas y columnas > > **np.vstack()**: Para apilar 2 arrays verticlalemente > > **np.hstack()**: Para apilar 2 arrays horizontalmente
- **.flatten()**: Copia un Array a un Array de 1 dimension

Funcion reshape() y T Modifica un Array en otro tipo de Array

Puedes generar Arrays de 1 dimension (vectores) a raiz de Arrays de mas dimensiones (matrices), viceversa,...

Caracteristicas:

- *a* indica el Array a modificar
- *newshape* la forma que sera *int* o una tupla de *int*.

Se puede utilizar de 2 formas:

- Invocandolo como metodo de la libreria
nombre array.reshape()
- Invocandolo como “metodo propio” (**metodo asociado a un objeto**) en el Array
ndarray.reshape
- Tambien podemos utilizar el metodo **T** que sirve para **transponer filas y columnas**
“nombre array”.T

```
[21]: import numpy as np

my_array = np.arange(1,10,1)
my_matriz = my_array.reshape(3,3)
print(my_matriz)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
[11]: import numpy as np

my_array = np.arange(1,10,1)
my_matriz = my_array.T
print(my_matriz)
```

```
[1 2 3 4 5 6 7 8 9]
```

Funciones para modificar varios arrays: vstack y hstack Modifica dos Arrays en otro tipo de Array

Los arrays que se apilan han de poder apilarse juntos, que sus formas matriciales lo permitan

No los mezcla, mantiene el orden de los arrays y **concatena** uno con otro.

Este metodo nos permite **anexionar arrays a otros que dispongan de dimensiones distintas**

Podemos anexionar vectores en fila a arrays de 2 dimensiones o mas o podemos añadir vectores en columna a arrays de 2 dimensiones o mas

Se puede utilizar para apilarlos vertical u horizontalmente:

- Para apilarlos verticalmente

`np.vstack(("nombre array 1" , "nombre array 2"))`

- Para apilarlos horizontalmente

`np.hstack(("nombre array 1" , "nombre array 2"))`

- Para anexionar vectores a matrices **como filas**

`np.vstack(("nombre array matriz" , "nombre array vector fila"))`

- Para anexionar vectores a matrices **como columnas**

`np.column_stack(("nombre array matriz" , "nombre array vector columna"))`

```
[25]: import numpy as np
```

```
array2 = np.arange(2,11).reshape(3,3)
array3 =np.arange(3,12).reshape(3,3)
print(array2)
print(array3)
```

```
[[ 2  3  4]
 [ 5  6  7]
 [ 8  9 10]]
[[ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

```
[26]: import numpy as np
```

```
array2 = np.arange(2,11).reshape(3,3)
array3 =np.arange(3,12).reshape(3,3)
print(np.vstack((array2, array3)))
```

```
[[ 2  3  4]
 [ 5  6  7]
 [ 8  9 10]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

```
[27]: import numpy as np

array2 = np.arange(2,11).reshape(3,3)
array3 =np.arange(3,12).reshape(3,3)
print(np.hstack((array2, array3)))
```

```
[[ 2  3  4  3  4  5]
 [ 5  6  7  6  7  8]
 [ 8  9 10  9 10 11]]
```

```
[33]: import numpy as np

array_vector = np.ones(3)
array2 = np.arange(2,11).reshape(3,3)
array3 =np.arange(3,12).reshape(3,3)
print(np.vstack((array2, array_vector)))
```

```
[[ 2.  3.  4.]
 [ 5.  6.  7.]
 [ 8.  9. 10.]
 [ 1.  1.  1.]]
```

```
[31]: import numpy as np

array_vector = np.ones(3)
array2 = np.arange(2,11).reshape(3,3)
array3 =np.arange(3,12).reshape(3,3)
print(np.column_stack((array2, array_vector)))
```

```
[[ 2.  3.  4.  1.]
 [ 5.  6.  7.  1.]
 [ 8.  9. 10.  1.]]
```

Funcion *flatten()* Crea una copia de un Array, con todos sus elementos a un Array de 1 dimension (vector)

Por tanto generar Arrays de 1 dimension (vectores) a raiz de Arrays de mas dimensiones (matrices)

nombre array.*flatten()*

Esta función permite ademas utilizar la expresion “**F**” que aplanla la matriz para hacer un vector **que se ordena por orden de columnas y no de filas** (es decir, comprueba las columnas que hay en cada fila y va poniendo en orden los valores de la columna 1, luego los siguientes digitos de la 2º... sin tener en cuenta las filas)

nombre array“*flatten("F")*

```
[9]: import numpy as np

array2 = np.random.random(10)
```

```
matrix_array2 = array2.reshape(5,2)
flatten_array2 = matrix_array2.flatten()
print(flatten_array2)
```

```
[0.66167845 0.97473746 0.21371356 0.29088149 0.65988842 0.37608317
 0.63789467 0.00190158 0.80174367 0.78797416]
```

```
[10]: import numpy as np

array2 = np.random.random(10)
matrix_array2 = array2.reshape(5,2)
flatten_array2 = matrix_array2.flatten("F")
print(flatten_array2)
```

```
[0.19421578 0.20823632 0.53312045 0.30826327 0.99753985 0.95968277
 0.66191766 0.81106886 0.8917687 0.00758886]
```

Funciones de division de arrays Seran aquellos metodos que permitiran generar otro array distinto en base a uno ya existente, escogiendo los elementos concretos.

- ***np.split()***: Parte un array en multiples arrays, en partes iguales.
- ***np.array_split()***: Parte un array en multiples arrays, de diferentes formas.
- ***np.hsplit()***: Corta un Array horizontalmente
- ***np.vsplit()***: Corta un Array verticalmente

Funciones *np.split()* y *np.array_split()* Permite obtener un array cortado en *varios arrays*

Si el array es de forma ([]) - Te permite separar las filas que componen un array en varios arrays

Estos podran ser:

- Uniformes: Arrays resultado con los elementos proporcionales
- Irregulares: Arrays resultado con distinto numero de elementos

Para generar Arrays uniformes utilizaremos *np.split()*

Es un metodo que intenta dividir proporcionalmente un array, sino dara error

np.split("nombre array", "nº de trozos o arrays resultado")

Para generar Arrays Irregulares utilizaremos *np.array_split()*

Es un metodo que intenta dividir proporcionalmente un array, y sino, organiza los datos para poder dividirlo en varios array.

np.array_split("nombre array", "nº de trozos o arrays resultado")

Se considera un *split normal + excepcion*

```
[12]: import numpy as np

array1 = np.array([[2,53,56,7,8,34] , [432,57,8,97,2,67]])
```

```
corte_array = np.split(array1, 1)
print(corte_array)
```

```
[array([[ 2, 53, 56,  7,  8, 34],
        [432, 57,  8, 97,  2, 67]])]
```

```
[16]: import numpy as np
```

```
array2 = np.arange (2, 10)
corte_array = np.split(array2, 2)
print(corte_array)
```

```
[array([2, 3, 4, 5]), array([6, 7, 8, 9])]
```

```
[10]: import numpy as np
```

```
array2 = np.arange (2, 10)
print(array2)
corte_array = np.array_split(array2, 6)
print(corte_array)
```

```
[2 3 4 5 6 7 8 9]
```

```
[array([2, 3]), array([4, 5]), array([6]), array([7]), array([8]), array([9])]
```

Funciones *np.hsplit* y *np.vsplit* Permite obtener un array cortado en *varios arrays proporcionales*

Los nº de arrays resultado han de poder realizarse, que sus formas matriciales **permitan dividirse**

No los mezcla, mantiene el orden de los arrays y **los separa justo en el punto de corte**

Se puede indicar los puntos de corte mediante tuplas, **sino es posible realizar el corte**

Este metodo nos permite **fraccionar un mismo array x veces**

Se puede utilizar para cortar vertical u horizontalmente:

- Para apilarlos verticalmente

np.vsplit("nombre array" , "nº de trozos o arrays resultado")

- Para apilarlos horizontalmente

np.hsplit("nombre array" , "nº de trozos o arrays resultado")

```
[42]: import numpy as np
```

```
array4 = np.arange(1,15).reshape (2,7)
corte_array = np.vsplit(array4, 2)
print(corte_array[0],corte_array[1] )
```

```
[[1 2 3 4 5 6 7]] [[ 8  9 10 11 12 13 14]]
```

```
[46]: import numpy as np

array4 = np.arange(1,15).reshape (2,7)
corte_array = np.hsplit(array4, 7)
print(corte_array)
```

```
[array([[1],
        [8]]), array([[2],
        [9]]), array([[ 3],
        [10]]), array([[ 4],
        [11]]), array([[ 5],
        [12]]), array([[ 6],
        [13]]), array([[ 7],
        [14]])]
```

1.4 Arrays: ¿Como se crean?

1.4.1 Crear Arrays de una lista

Se pueden crear Arrays desde una lista ya existente, utilizando el metodo *np.array()*

import numpy as np

“nombre array”= np.array(“nombre la lista”)

```
[19]: import numpy as np

my_list = [1,4,12]
my_array = np.array(my_list)
print(my_array)
```

```
[ 1  4 12]
```

Como hemos visto una diferencia entre las listas y los array es que son datos homogeneos de modo que **¿que pasa si intentamos hacer un array de una lista con varios tipos de datos?**

Respuesta: **Se generara un array de un tipo de dato que permita coexistir los datos**

Si por ejemplo existen datos *int* y *str* en la lista, el array se creara con datos *str*

Si por ejemplo existen datos *int* y *float* en la lista, el array se creara con datos *float*

```
[20]: import numpy as np

my_list = [1,4,"cosa"]
my_array = np.array(my_list)
print(my_array)
```

```
['1' '4' 'cosa']
```



```
[ 1.    4.  45.7]
```

```
[25]: import numpy as np

my_array = np.random.random(3)
print(my_array)
```

```
[0.89731989 0.45705595 0.46621171]
```

TIP: Solo se puede utilizar con datos *int* o *float* nunca *str* o *dara* error.

```
[26]: import numpy as np

my_array = np.ones("casa")
print(my_array)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[26], line 3
      1 import numpy as np
----> 3 my_array = np.ones("casa")
      4 print(my_array)

File ~\anaconda3\Lib\site-packages\numpy\_core\numeric.py:206, in ones(shape, dtype, order, device, like)
    201 if like is not None:
    202     return _ones_with_like(
    203         like, shape, dtype=dtype, order=order, device=device
    204     )
--> 206 a = empty(shape, dtype, order, device=device)
    207 multiarray.copyto(a, 1, casting='unsafe')
    208 return a

TypeError: 'str' object cannot be interpreted as an integer
```

Crear matrices o Arrays de más dimensiones (*np.ones()*, *np.zeros()*) Para hacer Arrays de más dimensiones (matrices):

```
import numpy as np

"nombre array"= np.ones/zeros(tupla("numero de dimensiones",
"numero de filas" , "elementos/scalars por fila",), "tipo int o float")
```

```
[27]: import numpy as np

my_array = np.ones((2,4,12), int)
print(my_array)
```

```
[[[1 1 1 1 1 1 1 1 1 1 1 1]
  [1 1 1 1 1 1 1 1 1 1 1 1]
```

```
[1 1 1 1 1 1 1 1 1 1 1 1]
[1 1 1 1 1 1 1 1 1 1 1 1]]
```

```
[[1 1 1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1 1 1]]]
```

Crear matrices *normalizadas* o Arrays de más dimensiones con intervalos definidos
 Para hacer *matrices normalizadas* Arrays de **más dimensiones (matrices) con intervalos definidos**

Se seguira este proceso:

- 1º Se creara un vector con los valores que hayamos querido que tuvieran el intervalo necesario, utilizaremos *arange()*

```
import numpy as np
```

```
"nombre array"= np.arange("start","stop","steps")
```

- 2º Modificaremos el array, vector en este caso, para **que se convierta en una matriz** utilizaremos *reshape()*

```
import numpy as np
```

```
"nombre matriz"= "nombre array".reshape("numero de dimen-
siones", "numero de filas")
```

```
[30]: import numpy as np

my_array = np.arange(2,12,3)
print(my_array)
```

```
[ 2  5  8 11]
```

```
[34]: import numpy as np

my_array = np.arange(2,12,3)
my_matriz = my_array.reshape(4,1)
print(my_matriz)
```

```
[[ 2]
 [ 5]
 [ 8]
 [11]]
```

TIP: Solo se podra cambiar a una forma de matriz que con **la cantidad de datos que existen se pueda dar** o sino generara un error

```
[35]: import numpy as np
```

```
my_array = np.arange(2,12,3)
my_matriz = my_array.reshape(5,2)
print(my_matriz)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[35], line 4
      1 import numpy as np
      3 my_array = np.arange(2,12,3)
----> 4 my_matriz = my_array.reshape(5,2)
      5 print(my_matriz)
```

ValueError: cannot reshape array of size 4 into shape (5,2)